

Jogo da Forca: Aplicação Distribuída Cliente-Servidor

João Pedro Becer da Cunha - 16987046, Layla Silva Nogueira - 15468671, Wender de Castro - 16834622

1. Proposta

O trabalho prático consiste no desenvolvimento de uma aplicação distribuída cliente-servidor que implementa o jogo da forca em modo multiplayer, executado via terminal e utilizando comunicação por sockets TCP.

A aplicação permite que múltiplos jogadores se conectem simultaneamente a um servidor e participem de uma mesma partida. O servidor é responsável por manter o estado central do jogo, incluindo a palavra secreta, as letras já tentadas, o número de erros e o turno atual, além de coordenar a interação entre os jogadores.

Os jogadores realizam tentativas de forma alternada, de acordo com o turno estabelecido. A cada jogada, o servidor atualiza o estado da partida e transmite o resultado aos clientes conectados, permitindo que todos acompanhem o andamento do jogo. A aplicação também apresenta o desenho da forca, atualizado conforme os erros são cometidos.

A partida é encerrada quando a palavra é descoberta ou quando o limite de tentativas é atingido. Dessa forma, o EP propõe a implementação de um jogo multiplayer distribuído, utilizando a comunicação em rede para coordenar diferentes clientes a partir de um servidor central.

2. Decisões de implementação

- **Linguagem e padrão (C++17):** Usamos C++ porque já tínhamos familiaridade com a linguagem e porque ela dá um controle mais direto sobre sockets e chamadas de sistema, sem esconder tanto o funcionamento da rede.
- **Arquitetura (cliente-servidor):** Escolhemos cliente-servidor porque o jogo tem um estado só (palavra, tentativas, turno) e fica mais simples manter tudo centralizado no servidor. Assim não precisamos sincronizar vários clientes entre si, e o servidor decide de quem é a vez.
- **Protocolo de transporte (TCP):** Usamos TCP (`SOCK_STREAM`) porque precisamos de confiabilidade. Se um palpite ou uma atualização de estado se perder, o jogo quebra. Fazer isso sobre UDP daria muito mais trabalho (timers, ACKs, controle de sequência) e a velocidade extra não compensa para a quantidade pequena de dados.

- **Concorrência:** O servidor cria uma thread para cada cliente. As partes compartilhadas (lista de clientes, estado do jogo) são protegidas com `std::mutex` para evitar condição de corrida.
- **Dependências:** Não usamos bibliotecas externas. Roda em Linux/macOS (POSIX) e também no WSL.

3. Protocolo

O protocolo roda sobre TCP e foi feito para ser simples.

3.1. Sintaxe

As mensagens têm um cabeçalho fixo e um payload de tamanho variável:

1 byte	2 bytes	N bytes
tipo	tamanho (N)	payload

- **Tipo (1 byte):** identifica o tipo da mensagem (`uint8_t`).
- **Tamanho (2 bytes):** `uint16_t` em *network byte order* (big-endian). Usamos `htons()` e `ntohs()` para converter.
- **Payload (N bytes):** dados da mensagem, sem terminador nulo.
- **Confiabilidade:** como o TCP é um fluxo de bytes e pode fragmentar os dados, implementamos as funções `enviarTudo` e `receberTudo`, que ficam em loop com `send()` e `recv()` até enviar ou receber todos os bytes esperados.

3.2. Semântica

O protocolo define 5 tipos de mensagens, declaradas no enum `TipoMsg` em `protocol.hpp`:

- **LOGIN (0x01):**
 - **Direção:** cliente → servidor.
 - **Payload:** `std::string` com o nome do jogador.
 - **Comportamento:** o servidor registra o cliente nas listas `clients` e `names` e responde com uma mensagem `STATE` contendo o estado inicial do jogo.
- **GUESS (0x02):**
 - **Direção:** cliente → servidor.
 - **Payload:** `std::string` com o chute (uma letra ou a palavra completa).

- **Comportamento:** o servidor confere se é a vez daquele jogador (`actualPlayer`). Se for, processa o chute chamando `jogo.input()` (para letras) ou `jogo.tentarPalavra()` (para palavras). Se o jogo não acabou, passa a vez e manda `STATE` para todos. Se acabou, manda `END_GAME` para todos.
- **STATE (0x03):**
 - **Direção:** servidor → clientes.
 - **Payload:** `std::string` com o tabuleiro atual (palavra oculta, tentativas, desenho da força e o nome de quem joga agora).
 - **Comportamento:** enviada via *broadcast* para todos os clientes sempre que o estado do jogo muda (depois de um `LOGIN` ou de um `GUESS` válido).
- **END_GAME (0x04):**
 - **Direção:** servidor → clientes.
 - **Payload:** `std::string` com a mensagem de vitória/derrota e o estado final.
 - **Comportamento:** enviada via *broadcast* quando `jogo.acabou()` retorna verdadeiro. Depois disso, o servidor inicia uma thread para esperar uma nova palavra do administrador.
- **LOGOUT (0x05):**
 - **Direção:** cliente → servidor.
 - **Payload:** vazio.
 - **Comportamento:** o servidor remove o cliente das listas, fecha o socket e ajusta o `actualPlayer` se o jogador que saiu estava na vez.

3.3. Gramática

1. **Conexão:** o cliente conecta via TCP, envia `LOGIN` com o nome e recebe de volta um `STATE`.
2. **Ciclo de jogo:**
 - O servidor controla o `actualPlayer` (índice de quem joga agora).
 - Só o cliente da vez envia `GUESS`. O servidor valida o chute.
 - Se o jogo não acabou, o servidor passa a vez e manda `STATE` para todos.
 - Se o jogo acabou, o servidor manda `END_GAME` para todos e espera uma nova palavra.
3. **Encerramento:** o cliente manda `LOGOUT` e o servidor remove a conexão da lista.

4. Leitura do código

A estrutura de pastas do projeto é:

```
EP_Redes/
|-- bin/                # Binarios gerados pela compilacao
|-- include/            # Arquivos de cabecalho (.hpp)
|   |-- client.hpp
|   |-- forca.hpp
|   |-- protocol.hpp
|   |-- server.hpp
|   '-- utils.hpp
|-- src/                #Codigo-fonte (.cpp)
|   |-- client.cpp
|   |-- client_main.cpp
|   |-- forca.cpp
|   |-- protocol.cpp
|   |-- server.cpp
|   |-- server_main.cpp
|   '-- utils.cpp
|-- Makefile
|-- README.md
'-- boneco.txt
```

- **Pontos de entrada (main):** a execução começa em `server_main.cpp` (cria o servidor, pede a palavra secreta e a porta) e `client_main.cpp` (cria o cliente, pede IP, porta e nome).
- **Camada de rede:** `protocol.hpp/cpp` cuida do envio e recebimento das mensagens. `server.hpp/cpp` e `client.hpp/cpp` gerenciam os sockets e as threads.
- **Lógica do jogo:** `forca.hpp/cpp` tem as regras do jogo e o desenho ASCII (também disponível em `boneco.txt`).
- **Utilitários:** `utils.hpp/cpp` contém apenas funções auxiliares de manipulação de strings: `compare`, `escondePalavra` e `revelaPalavra`.

5. Como usar a aplicação

Requisitos de Sistema: Requer sistema operacional POSIX (Linux, macOS ou WSL no Windows) e compilador C++17 com GNU Make

Compilação: Na raiz do projeto, execute no terminal 'make'
comando gera os executáveis `bin/server` e `bin/client`.

5.1. Passo a Passo de Execução e Conexão

- **Servidor** deve ser iniciado obrigatoriamente antes de qualquer cliente para abrir o socket de escuta: `./bin/server`
- Ao ser executado, o servidor aloca a porta de comunicação, carrega a palavra secreta e lê o arquivo de arte ASCII da forca (`boneco.txt`), permanecendo no aguardo das conexões dos jogadores.
- **Cliente:** `./bin/client`
- Solicita obrigatoriamente três dados no terminal :
 - **Endereço IP do Servidor:** O usuário digita `127.0.0.1` (ou `localhost`) para testes na mesma máquina; ou insere o IP local da máquina servidor (obtido executando o comando `ip addr` no terminal do servidor) para conexões entre máquinas distintas na mesma rede Wi-Fi/local.
 - **Porta de Comunicação:** O número da porta em que o servidor está escutando.
 - **Nome do Jogador (Nickname):** A identificação do usuário na partida.

5.2. Dinâmica e Uso do Jogo

- **Entrada:** Ao fornecer os dados, o cliente envia um pacote `LOGIN` e entra na partida
- **Interface:** A tela do terminal exibe a máscara da palavra (ex.: `_ E D E _`), a forca em arte ASCII (`boneco.txt`) atualizada a cada erro e o jogador da vez
- **Interação:** Apenas o jogador no turno ativo digita o palpite (letra ou palavra inteira), enviado via mensagem `GUESS`. Palpites fora do turno são ignorados pelo servidor.
- **Validação e Sincronização:** O servidor valida a jogada (funções `compare`, `escondePalavra` e `revelaPalavra` do `utils.cpp`) e transmite a mensagem `STATE` em *broadcast* para todos os clientes, atualizando os terminais simultaneamente

5.3. Encerramento da Aplicação

- **Finalização Automática:** O jogo é encerrado automaticamente quando a palavra secreta é adivinhada (vitória) ou quando o limite de erros é atingido com o preenchimento total da forca (derrota). O servidor transmite a mensagem do fim do jogo, exibe o resultado nos terminais e encerra a rodada.
- **Desconexão:** Caso um cliente encerre a sessão, uma mensagem é enviada para que o servidor encerre a *thread* daquele usuário, permitindo que a partida continue com os jogadores remanescentes.

6. Testes e resultados

A aplicação foi validada em múltiplas rodadas com diferentes palavras e cenários (vitória, limite de erros e palpites repetidos).

6.1. Ambiente de Testes, Conexão e Login

- **Servidor (Máquina 1):** Instância Linux (WSL2/Ubuntu) escutando na porta 2026/TCP
- **Cliente 1 (Máquina 1):** Jogador fictício conectada via '127.0.0.1:2026' (WSL2/Ubuntu) escutando na porta 2026/TCP
- **Cliente 2 (Máquina 2 - Rede Sem Fio):** Jogador fictício conectado via IP de rede local '192.168.68.101:2026'

```
layla@DESKTOP-NCM3BCR: ~/EP-REDES-USP
layla@DESKTOP-NCM3BCR:/mnt/c/Program Files/Autodesk/Inventor 2025/Bin$ cd ~/EP-REDES-USP
layla@DESKTOP-NCM3BCR:~/EP-REDES-USP$ ./bin/server
Digite a palavra desejada:
inventor
Digite a porta a ser aberta pelo servidor:
2026
SERVER::LISTENING_ON_PORT: 2026
SERVER::ACTUAL_WORD: inventor
SERVER::NEW_CLIENT: 4
SERVER::CLIENT_IP: 127.0.0.1
SERVER::NEW_CLIENT: 5
SERVER::CLIENT_IP: 172.20.176.1
SERVER::RECEIVED_LOGIN: joao
SERVER::RECEIVED_LOGIN: maria

Selecionar layla@DESKTOP-NCM3BCR: ~/EP-REDES-USP
layla@DESKTOP-NCM3BCR:/mnt/c/Program Files/Autodesk/Inventor 2025/Bin$ cd ~/EP-REDES-USP
layla@DESKTOP-NCM3BCR:~/EP-REDES-USP$ ./bin/client
Digite o IP:
127.0.0.1
Digite a porta:
2026
=====INICIALIZING CLIENT | ID:-1=====
Digite o nome a ser chamado:
joao
=====STATE=====
```

Maquina 1 - Inicialização do servidor com a palavra 'inventor' e conexão simultânea dos clientes



Palpites com a palavra secreta 'CAIXA'

6.3. Palpite Correto e Sincronização

Ilustra o envio da mensagem 'GUESS' contendo a letra correta, por exemplo: A palavra correta é CAIXA, e tem a entrada da letra 'C'. O servidor valida o palpite, atualiza o estado interno do jogo e transmite o pacote 'STATE' via broadcast, revelando as ocorrências da letra (C A I _ A) simultaneamente nos dois terminais.



Palpites com a palavra secreta 'CAIXA'

Palpites com a palavra secreta 'CAIXA'

Exibe o momento em que o jogador envia o palpite correto com a palavra completa, por exemplo: " CAIXA ". O servidor valida a vitória, encerra o ciclo da partida e dispara a mensagem para todos os participantes, apresentando a notificação de vitória no terminal de vencedor e revelando a palavra secreta em ambos os clientes. Quando encerra a partida após o esgotamento do limite de tentativas dos participantes, o servidor valida o acúmulo de erros, encerra a rodada e dispara a mensagem para todos os clientes conectados, exibindo a notificação "Você perdeu" e "Acabaram todas as tentativas restantes" nos terminais.

```
====GAME OVER====
Acabaram as tentativas restantes..._____
tentativas:
```

VOCÊ
PERDEU

6.5. Limitações

- Durante a inicialização da aplicação cliente, observa-se a exibição temporária da identificação do jogador como "ID: -1".
- Durante o decorrer de uma partida, o console do servidor continua capturando entradas do teclado. Caso ocorram digitações acidentais no terminal do servidor enquanto os clientes jogam, esses caracteres permanecem retidos no buffer do sistema operacional. Ao término da rodada, quando o servidor solicita a definição de uma nova palavra secreta, a leitura da entrada padrão consome imediatamente esse "lixo" de memória acumulado.

7. Fontes

- [1] KUROSE, J. F.; ROSS, K. W. **Redes de Computadores e a Internet: Uma Abordagem Top-Down**. 6. ed. São Paulo: Pearson Education do Brasil, 2013.
- [2] MARGI, C. B. et al. **IT-SDN: Improved Architecture for Software-Defined Wireless Sensor Networks**. Repositório da Produção USP, 2017. Disponível em: <https://repositorio.usp.br/item/003036149> e <https://sites.google.com/usp.br/cintia/it-sdn>.
- [3] IPOLITO, R. **Programação com Sockets**. Faculdade de Tecnologia, Universidade Estadual de Campinas (UNICAMP). Disponível em: https://www.ic.unicamp.br/~ripolito/peds/st564/material/aula_socket.pdf.
- [4] GEEKSFORGEES. **Socket Programming in C/C++**. Disponível em: <https://www.geeksforgeeks.org/cpp/socket-programming-in-cpp/>.
- [5] KELLY, S. **Creating a TCP Server in C++ (Linux)**. YouTube, 2018. Disponível em: <https://youtu.be/cNdlrbZSkyQ>.
- [6] JOHANNES 4GNU.LINUX. **GNU/Linux & Ethernet: Writing a simple TCP socket Server in Linux with C**. YouTube, 2025. Disponível em: <https://youtu.be/s3d1YSu70CU>.
- [7] KERRISK, M. **The Linux Programming Interface / Linux Programmer's Manual** (Man pages: socket(2), bind(2), listen(2), accept(2), send(2), recv(2)).